syndicate

Encode XML and generate Atom feeds, RSS feeds and sitemaps

Version 0.1.0 · Typst 0.15.0 or later
https://github.com/bfeitknecht/syndicate

syndicate encodes Typst values to XML. It generates Atom 1.0 feeds, RSS 2.0 feeds and XML sitemaps for websites and blogs that you make with Typst. It is written in pure Typst and does not use plugins. The license is the MIT License.

Contents

1 Introduction 3
2 Start 4
2.1 One data source 4
2.2 Write the files 4
2.3 Text, dates and time zones 5
3 XML 6
3.1 Encode XML 6
3.2 Decode XML 6
4 Feeds 8
4.1 Atom 8
4.2 RSS 9
5 Sitemaps 10
6 Extend documents 11
7 Reference 12
7.1 xml 12
7.2 atom 16
7.3 rss 23
7.4 sitemap 30
7.5 Dates 33

1 Introduction

Typst can export HTML. Thus, you can use Typst to make static websites. But a website usually has more than pages. Readers subscribe to **feeds**, and search engines read **sitemaps**. Feeds and sitemaps are XML documents. Typst can read XML, but it cannot write XML.

syndicate has four modules that write XML:

Module	Contents
`xml`	`encode` and `decode` for all XML, and functions that make nodes
`atom`	Atom 1.0 feeds (RFC 4287): `feed`, `entry`, `author`, `tag-uri`
`rss`	RSS 2.0 feeds: `feed`, `item`, `enclosure`
`sitemap`	XML sitemaps: `urlset` and `url`, and sitemap indexes with `index`

The package also exports the date functions `rfc3339` and `rfc2822`. Import the functions that you need:

```
#import "@preview/syndicate:0.1.0": xml, atom, rss, sitemap
```

Each document function returns the complete XML as a string. When you import `xml`, it hides the `xml()` reader of Typst in your file. To read XML, use `xml.decode` or `std.xml`. `xml.decode` uses the reader of Typst.

2 Start

2.1 One data source

Keep the data of your posts in one location. Then generate all formats from this data:

```
#import "@preview/syndicate:0.1.0": atom, rss, sitemap

#let site = "https://example.com/"
#let posts = (
  (
    title: "Hello, world",
    slug: "hello-world",
    date: datetime(year: 2026, month: 7, day: 1),
    summary: [My _first_ post],
  ),
  (title: "Second post", slug: "second", date: "2026-07-14T09:30", summary: "More things"),
)
#let url(p) = site + "blog/" + p.slug + "/"

#let feed = atom.feed(
  title: "My Blog",
  link: site,
  self-link: site + "feed.xml",
  authors: "Basil",
  entries: posts.map(p => atom.entry(
    p.title,
    link: url(p),
    published: p.date,
    summary: p.summary,
  )),
)

#let rss-feed = rss.feed(
  title: "My Blog",
  link: site,
  description: "Things I write",
  self-link: site + "rss.xml",
  items: posts.map(p => rss.item(
    p.title,
    link: url(p),
    published: p.date,
    description: p.summary,
  )),
)

#let map = sitemap.urlset(site, posts.map(p => sitemap.url(url(p), lastmod: p.date)))
```

2.2 Write the files

Usually, a Typst document makes one output. There are two procedures to get the XML files.

With bundle export. Bundle export is an experimental output format of Typst 0.15. It writes many files. If you have HTML pages, it writes the XML files together with them:

```
#asset("feed.xml", feed)
#asset("rss.xml", rss-feed)
#asset("sitemap.xml", map)
```

```
typst compile --features bundle --format bundle blog.typ site/
```

With `typst eval`. Put the strings into `metadata`. Then get them with `typst eval` on the command line. In Typst 0.15, the output of `eval` is JSON. Thus, use `jq` to get the string from the JSON:

```
#metadata(feed) <feed>
```

```
typst eval --in blog.typ 'query(<feed>).first().value' | jq -j . > feed.xml
```

2.3 Text, dates and time zones

Text fields accept strings. They also accept Typst content that contains only simple markup, for example text, spaces, smart quotes, `*strong*`, `_emph_`, `raw` , links and underline. Feeds can contain only plain text. Thus, the functions remove the markup. For example, `[My _first_ post]` changes to `My first post`. Some content has no correct plain-text form, for example equations, footnotes and images. For this content, the functions stop with an error that shows the field. They do not remove content without a message. If you are not sure, use a string.

Dates are `datetime` values or ISO 8601 strings without a time zone: `"2026-07-01"`, `"2026-07-01T14:30"` or `"2026-07-01T14:30:00"`. Strings are easy to use with data from YAML or JSON files. If a date has no time, the functions use `00:00:00`. In sitemaps, a date without a time also has no time in the output.

Time zones apply to a full document. Set them with `tz`, for example `"Z"` (the default), `"+01:00"` or `"-0530"`. A `datetime` of Typst has no time zone. Thus, the functions use `tz` for all dates in a document.

```
rfc3339("2026-07-01", tz: "+02:00")
```

↓

```
2026-07-01T00:00:00+02:00
```

3 XML

3.1 Encode XML

`xml.encode` serializes a tree of nodes. An element is a dictionary with a `tag` and optional `attrs` and `children`. The `xml()` reader of Typst returns the same structure. Text is a string, a number or simple content. `xml.encode` escapes text automatically.

```
xml.encode(
  (
    tag: "bookstore",
    children: (
      xml.comment(" inventory "),
      (
        tag: "book",
        attrs: (id: "bk101", category: "programming", discontinued: none),
        children: (
          (tag: "title", attrs: ("xml:lang": "en"), children: ("XML & Typst",)),
          (tag: "price", children: (44.95,)),
        ),
      ),
    ),
  ),
  pretty: true,
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <!-- inventory -->
  <book id="bk101" category="programming">
    <title xml:lang="en">XML &amp;amp; Typst</title>
    <price>44.95</price>
  </book>
</bookstore>
```

- **Attributes** keep their order. `xml.encode` does not write attributes that are `none`. It converts numbers and booleans to text.
- **Children** can contain `none` and nested arrays. `xml.encode` ignores `none` and flattens the arrays. Thus, you can use `if` expressions and `.map(..)` in the list of children.
- **`pretty`** indents only elements that contain only elements. `xml.encode` does not add whitespace adjacent to text. Thus, text does not change.
- **Namespace prefixes** are part of the name, for example `(tag: "atom:link")`. The key `namespace` contains a namespace URI. Refer to Section 3.2.
- **Incorrect input causes an error.** Examples are incorrect names, characters that XML prohibits, `--` in comments, more than one root element and keys with spelling errors.

For the other node types, use `xml.comment`, `xml.cdata` and `xml.pi`. To add markup that is already serialized, use `xml.raw`. `xml.encode` writes this markup without changes.

3.2 Decode XML

`xml.decode` parses XML into the same structure. It uses the XML reader of Typst. It accepts a string, bytes or a `path(..)` to a file. For a file, use `path`, not only the file name. Then Typst finds the file relative to your document, not relative to this package.

```
let doc = xml.decode("<list><item>a</item><item>b</item></list>").first()
let items = doc.children.filter(c => type(c) == dictionary)
xml.encode(
  (..doc, children: items + ((tag: "item", children: ("c",)),)),
  declaration: false,
)
```

↓

```
<list><item>a</item><item>b</item><item>c</item></list>
```

A decoded element has the key `namespace` with its namespace URI. `xml.encode` changes the URI back to `xmlns` declarations. Thus, the encoded document keeps the namespaces:

```
xml.encode(
  xml.decode("<r xmlns='urn:a' xmlns:b='urn:b'><b:x/><y/></r>"),
  declaration: false,
)
```

↓

```
<r xmlns="urn:a"><x xmlns="urn:b"/><y/></r>
```

`xml.decode` has two limits, because the reader of Typst has them:

- The reader removes comments and processing instructions. It puts empty placeholder elements in their positions. `xml.decode` removes these placeholders. If the output of `std.xml` contains placeholders, `xml.encode` stops with an error.
- The reader removes prefixes. For example, `<media:thumbnail>` changes to `thumbnail` with the media namespace URI, and `xml:lang` changes to `lang`.

4 Feeds

Almost all feed readers can read Atom and RSS. The specification of Atom is more precise. RSS is older, and podcasts use it more frequently. To publish both formats, you need only one more function call.

4.1 Atom

```
atom.feed(
  title: "My Blog",
  link: "https://example.com/",
  self-link: "https://example.com/feed.xml",
  id: atom.tag-uri("example.com", "2026-01-01", "blog"),
  authors: atom.author("Basil", email: "basil@example.com"),
  tz: "+01:00",
  entries: atom.entry(
    "Hello, world",
    link: "https://example.com/blog/hello/",
    id: atom.tag-uri("example.com", "2026-07-01", "blog/hello"),
    published: "2026-07-01",
    updated: "2026-07-03T14:30",
    summary: [My _first_ post],
    content: "<p>Welcome to my <em>new</em> blog.</p>",
    categories: ("meta", (term: "typst", label: "Typst")),
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <title>My Blog</title>
  <link rel="alternate" type="text/html" href="https://example.com/"/>
  <link rel="self" type="application/atom+xml" href="https://example.com/feed.xml"/>
  <id>tag:example.com,2026-01-01:blog</id>
  <updated>2026-07-03T14:30:00+01:00</updated>
  <author>
    <name>Basil</name>
    <email>basil@example.com</email>
  </author>
  <entry>
    <title>Hello, world</title>
    <link rel="alternate" type="text/html" href="https://example.com/blog/hello/"/>
    <id>tag:example.com,2026-07-01:blog/hello</id>
    <updated>2026-07-03T14:30:00+01:00</updated>
    <published>2026-07-01T00:00:00+01:00</published>
    <category term="meta"/>
    <category term="typst" label="Typst"/>
    <summary>My first post</summary>
    <content type="html">&lt;p&gt;Welcome to my &lt;em&gt;new&lt;/em&gt; blog.&lt;/p&gt;</
content>
  </entry>
</feed>
```

Each feed and each entry must have a permanent, unique `id`. The default is the `link`. But the link changes when you move your website. `atom.tag-uri` makes identifiers that do not change. Each entry must have an author. Set the authors in the feed or in each entry. If `content` is a string, it is HTML source. The function escapes it as Atom specifies. The function converts Typst content to plain text.

4.2 RSS

```
rss.feed(
  title: "My Podcast",
  link: "https://example.com/",
  description: "Conversations about typesetting",
  self-link: "https://example.com/rss.xml",
  language: "en-us",
  items: rss.item(
    "Episode 1",
    link: "https://example.com/podcast/1/",
    published: "2026-07-08T06:00",
    author: (name: "Basil", email: "basil@example.com"),
    enclosure: rss.enclosure(
      "https://example.com/podcast/1.mp3",
      length: 24986239,
      type: "audio/mpeg",
    ),
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0" xmlns:atom="http://www.w3.org/2005/Atom">
  <channel>
    <title>My Podcast</title>
    <link>https://example.com/</link>
    <description>Conversations about typesetting</description>
    <atom:link href="https://example.com/rss.xml" rel="self" type="application/rss+xml"/>
    <language>en-us</language>
    <lastBuildDate>Wed, 08 Jul 2026 06:00:00 +0000</lastBuildDate>
    <item>
      <title>Episode 1</title>
      <link>https://example.com/podcast/1/</link>
      <author>basil@example.com (Basil)</author>
      <enclosure url="https://example.com/podcast/1.mp3" length="24986239" type="audio/mpeg"/>
      <guid isPermaLink="true">https://example.com/podcast/1/</guid>
      <pubDate>Wed, 08 Jul 2026 06:00:00 +0000</pubDate>
    </item>
  </channel>
</rss>
```

In RSS, `<author>` must contain an email address. If an author has only a name, `rss.item` writes the name as `<dc:creator>`. If the feed uses the `atom`, `content` or `dc` namespace, `rss.feed` declares it automatically.

5 Sitemaps

A sitemap lists the pages of a website for search engines. The pages are `sitemap.url(..)` entries or URL strings. Give them as arguments or arrays.

```
sitemap.urlset(
  sitemap.url("https://example.com/", changefreq: "daily", priority: 1),
  sitemap.url(
    "https://example.com/about/",
    lastmod: "2026-05-01",
    alternates: (de: "https://example.com/de/about/"),
  ),
  "https://example.com/imprint/",
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" xmlns:xhtml="http://www.w3.org/
1999/xhtml">
  <url>
    <loc>https://example.com/</loc>
    <changefreq>daily</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://example.com/about/</loc>
    <lastmod>2026-05-01</lastmod>
    <xhtml:link rel="alternate" hreflang="de" href="https://example.com/de/about/"/>
  </url>
  <url>
    <loc>https://example.com/imprint/</loc>
  </url>
</urlset>
```

A sitemap can contain a maximum of 50 000 URLs. For a larger website, divide the list into more than one sitemap. Then list the sitemaps with `sitemap.index`.

6 Extend documents

All document functions accept `tz`, `stylesheet` and `pretty`:

- **`stylesheet`** adds an `<?xml-stylesheet?>` instruction. Then browsers show a page with styles, not the XML source. You can use XSL or CSS.
- **`pretty`** indents the output. The default is `true`.

`atom.feed`, `rss.feed` and `sitemap.urlset` also accept `namespaces`. These are more `(prefix: uri)` declarations on the root element. The feeds, their entries and `sitemap.url` accept `extra`. These are XML nodes that the function adds at the end of the element. With these two options, you can use extensions, for example Media RSS:

```
rss.feed(
  title: "Gallery",
  link: "https://example.com/",
  description: "Pictures",
  namespaces: (media: "http://search.yahoo.com/mrss/"),
  items: rss.item(
    "Sunset",
    link: "https://example.com/sunset/",
    extra: (tag: "media:content", attrs: (url: "https://example.com/sunset.jpg")),
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0" xmlns:media="http://search.yahoo.com/mrss/">
  <channel>
    <title>Gallery</title>
    <link>https://example.com/</link>
    <description>Pictures</description>
    <item>
      <title>Sunset</title>
      <link>https://example.com/sunset/</link>
      <guid isPermaLink="true">https://example.com/sunset/</guid>
      <media:content url="https://example.com/sunset.jpg"/>
    </item>
  </channel>
</rss>
```

7 Reference

7.1 xml

- escape()
- element()
- comment()
- cdata()
- pi()
- raw()
- encode()
- decode()

escape

Escapes text for XML character data. The function changes &, < and > to entity references.

```
xml.escape("Fish & <Chips>")
```

↓

```
Fish &amp; &lt;Chips&gt;
```

Parameters

```
escape(text: str content) -> str
```

text `str` or `content`

The text.

element

Makes an element node. The result is the same as the dictionary `(tag: tag, attrs: attrs, children: children)`.

```
xml.element("a", attrs: (href: "/"), "Home")
```

↓

```
(tag: "a", attrs: (href: "/"), children: ("Home",))
```

Parameters

```
element(
  tag: str,
  attrs: dictionary,
  ..children: str content dictionary array none
) -> dictionary
```

tag `str`

The element name. It can have a namespace prefix, for example `"atom:link"`.

attrs `dictionary`

The attributes. The function does not write attributes that are `none`.

Default: `(:)`

..children `str` or `content` or `dictionary` or `array` or `none`

The child nodes.

comment

Makes a comment node. `xml.encode` writes it as `<!--body-->`.

```
xml.encode(
  (tag: "p", children: (xml.comment(" draft "),)),
  declaration: false,
)
```

↓

```
<p><!-- draft --></p>
```

Parameters

`comment(body: str) -> dictionary`

body `str`

The comment text. It must not contain `--`, and it must not end with `-`.

cdata

Makes a CDATA section. `xml.encode` writes it as `<![CDATA[body]]>`. If the text contains `]]>`, the function divides the text into two sections. Thus, all text is safe.

```
xml.encode(
  (tag: "script", children: (xml.cdata("if (a < b) go()"),)),
  declaration: false,
)
```

↓

```
<script><![CDATA[if (a < b) go()]]></script>
```

Parameters

```
cdata(body: str) -> dictionary
```

body `str`

The text.

pi

Makes a processing instruction. `xml.encode` writes it as `<?target data?>`.

```
xml.encode(
  (xml.pi("xml-stylesheet", "href=\"style.xsl\""), (tag: "doc")),
  declaration: false,
)
```

↓

```
<?xml-stylesheet href="style.xsl"?><doc/>
```

Parameters

```
pi(
  target: str,
  data: str none
) -> dictionary
```

target `str`

The target, for example `"xml-stylesheet"`.

data `str` or `none`

The data of the instruction. It must not contain `?>`.

raw

Makes a node from markup that is already serialized. `xml.encode` writes the markup without changes. Make sure that the markup is well-formed.

```
xml.encode(
  (tag: "p", children: (xml.raw("<b>bold</b> &amp; raw"),)),
  declaration: false,
)
```

↓

```
<p><b>bold</b> &amp; raw</p>
```

Parameters

```
raw(body: str) -> dictionary
```

body `str`

The markup.

encode

Serializes a tree of nodes to an XML document.

An element is a dictionary `(tag: .., attrs: .., children: ..)`. You can leave out `attrs` and `children`. Text is a string, a number or simple content. The function escapes text automatically. It ignores children that are `none` and flattens nested arrays. Thus, you can use `if` and `.map(..)` in the list of children.

```
xml.encode(
  (
    tag: "book",
    attrs: (id: "bk101", lang: none),
    children: (
      (tag: "title", children: ("XML & Typst",)),
      (tag: "price", children: (44.95,)),
    ),
  ),
  pretty: true,
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<book id="bk101">
  <title>XML &amp;amp; Typst</title>
  <price>44.95</price>
</book>
```

Parameters

```
encode(
  root: dictionary array,
  pretty: bool,
  indent: int str,
  declaration: bool str
) -> str
```

root `dictionary` or `array`

The root element, or an array of top-level nodes. The array must contain one element. It can also contain comments and processing instructions.

pretty `bool`

If `true`, the function indents elements that contain only elements. It does not change text.

Default: `false`

indent `int` or `str`

The indentation for each level if `pretty` is `true`. Use a number of spaces or a string, for example `"\t"`.

Default: `2`

declaration `bool` or `str`

The XML declaration. Use `true` for `<?xml version="1.0" encoding="UTF-8"?>`, `false` for no declaration, or a string for a different declaration.

Default: `true`

decode

Parses XML into the node structure of `xml.encode`. The function uses the XML reader of Typst.

The function returns each element as `(namespace: .., tag: .., attrs: .., children: ..)`. The `namespace` is the namespace URI of the element. `xml.encode` changes it back to `xmlns` declarations. The reader removes comments, processing instructions and namespace prefixes.

```
xml.decode("<p>Hello <b>world</b></p>")
```

↓

```
(
  (
    namespace: none,
    tag: "p",
    attrs: (:),
    children: (
      "Hello ",
      (
        namespace: none,
        tag: "b",
        attrs: (:),
        children: ("world",),
      ),
    ),
  ),
)
```

Parameters

`decode(source: str bytes path) -> array`

source `str` or `bytes` or `path`

The XML as a string or bytes, or a `path(..)` to a file. For a file, use `path`, not only the file name. Then Typst finds the file relative to your document, not relative to this package.

7.2 atom

- tag-uri()
- author()

- entry()
- feed()

tag-uri

Makes a `tag:` URI (RFC 4151). Use it as a permanent, unique identifier for a feed or an entry. The identifier does not change when your URLs change.

```
atom.tag-uri("example.com", "2026-07-01", "blog/hello-world")
```

↓

```
tag:example.com,2026-07-01:blog/hello-world
```

Parameters

```
tag-uri(
  authority: str,
  date: datetime str,
  specific: str
) -> str
```

authority `str`

A domain name or an email address that you controlled on `date`.

date `datetime` or `str`

A date on which you controlled `authority`. Usually, this is the date when you made the feed or the entry.

specific `str`

A text that is unique for this authority and date.

author

Describes a person for the `authors` and `contributors` of feeds and entries. Where a function expects a person, you can also use a string. The function uses the string as the name.

Parameters

```
author(
  name: str content,
  email: str none,
  uri: str none
) -> dictionary
```

name `str` or `content`

The name of the person.

email `str` or `none`

An email address.

Default: `none`

uri `str` or `none`

A web page.

Default: `none`

entry

Describes an entry for `atom.feed(entries: ..)`.

```
atom.entry(
  "Hello, world",
  link: "https://example.com/blog/hello/",
  published: "2026-07-01",
  summary: [My _first_ post],
)
```

Parameters

```
entry(
  title: str content,
  id: str auto,
  link: str none,
  updated: datetime str auto,
  published: datetime str none,
  summary: str content none,
  content: str content none,
  content-type: str auto,
  authors: array,
  contributors: array,
  categories: array,
  rights: str content none,
  extra: array
) -> dictionary
```

title `str` or `content`

The title.

id `str` or `auto`

A permanent, unique identifier. The default is `link`. If your URLs can change, use `atom.tag-uri`.

Default: `auto`

link `str` or `none`

The URL of the web page of the entry. It is necessary if there is no `content`.

Default: `none`

updated `datetime` or `str` or `auto`

The date of the last important change. The default is `published`.

Default: `auto`

published `datetime` or `str` or `none`

The date of the first publication.

Default: `none`

summary `str` or `content` or `none`

A short summary in plain text.

Default: `none`

content `str` or `content` or `none`

The full content. A string is HTML source. The function escapes it as Atom specifies. The function converts content to plain text.

Default: `none`

content-type `str` or `auto`

The `type` of the content: `"html"`, `"text"` or a MIME type. The default is `"html"` for strings and `"text"` for content.

Default: `auto`

authors `array`

The authors, as strings or `atom.author(..)`.

Default: `()`

contributors `array`

The contributors, as strings or `atom.author(..)`.

Default: `()`

categories `array`

The categories, as strings or dictionaries `(term: .., scheme: .., label: ..)`.

Default: `()`

rights `str` or `content` or `none`

A copyright statement.

Default: `none`

extra `array`

More XML nodes. The function adds them at the end of the entry.

Default: `()`

feed

Generates an Atom 1.0 feed.

```
atom.feed(
  title: "My Blog",
  link: "https://example.com/",
  authors: "Basil",
  entries: atom.entry(
    "Hello, world",
    link: "https://example.com/blog/hello/",
    published: "2026-07-01",
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <title>My Blog</title>
  <link rel="alternate" type="text/html" href="https://example.com/"/>
  <id>https://example.com/</id>
  <updated>2026-07-01T00:00:00Z</updated>
  <author>
    <name>Basil</name>
  </author>
  <entry>
    <title>Hello, world</title>
    <link rel="alternate" type="text/html" href="https://example.com/blog/hello/"/>
    <id>https://example.com/blog/hello/</id>
    <updated>2026-07-01T00:00:00Z</updated>
    <published>2026-07-01T00:00:00Z</published>
  </entry>
</feed>
```

Parameters

```
feed(
  title: str content,
  subtitle: str content none,
  link: str,
  self-link: str none,
  id: str auto,
  updated: datetime str auto,
  authors: array str,
  contributors: array,
  categories: array,
  rights: str content none,
  icon: str none,
  logo: str none,
  generator: str dictionary none,
  language: str none,
  entries: array,
  tz: str,
  stylesheet: str none,
  namespaces: dictionary,
  extra: array,
  pretty: bool
) -> str
```

title `str` or `content`

The title of the feed. This value is necessary.

Default: `none`

subtitle `str` or `content` or `none`

A subtitle or a description.

Default: `none`

link `str`

The URL of the website. This value is necessary.

Default: `none`

self-link `str` or `none`

The URL of this feed. Set it if possible.

Default: `none`

id `str` or `auto`

A permanent, unique identifier. The default is `link`.

Default: `auto`

updated `datetime` or `str` or `auto`

The date of the last change. The default is the latest `updated` of the entries.

Default: `auto`

authors `array` or `str`

The authors, as strings or `atom.author(..)`. This value is necessary if an entry has no authors.

Default: `()`

contributors `array`

The contributors, as strings or `atom.author(..)`.

Default: `()`

categories `array`

The categories, as strings or dictionaries `(term: .., scheme: .., label: ..)`.

Default: `()`

rights `str` or `content` or `none`

A copyright statement.

Default: `none`

icon `str` or `none`

The URL of a small, square icon, for example a favicon.

Default: `none`

logo `str` or `none`

The URL of a larger logo with an aspect ratio of 2:1.

Default: `none`

generator `str` or `dictionary` or `none`

The software that generated the feed. Use a name or a dictionary `(name: .., uri: .., version: ..)`.

Default: `none`

language `str` or `none`

The language of the feed, for example `"en"`. The function writes it as `xml:lang`.

Default: `none`

entries `array`

The entries. Make them with `atom.entry(..)`.

Default: `()`

tz `str`

The time zone of all dates. Use `"Z"` for UTC, or an offset, for example `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet. Browsers apply it to the feed.

Default: `none`

namespaces `dictionary`

More namespace declarations `(prefix: uri)` for the elements in `extra`.

Default: `(:)`

extra `array`

More XML nodes. The function adds them at the end of the feed.

Default: `()`

pretty `bool`

If `true`, the function indents the output.

Default: `true`

7.3 rss

- enclosure()
- item()
- feed()

enclosure

Describes a media attachment for `rss.item(enclosure: ..)`, for example a podcast episode. All three values are necessary.

```
rss.enclosure(
  "https://example.com/episode-1.mp3",
  length: 24986239,
  type: "audio/mpeg",
)
```

Parameters

```
enclosure(
  url: str,
  length: int,
  type: str
) -> dictionary
```

url `str`

The URL of the file.

length `int`

The size of the file in bytes.

Default: `none`

type `str`

The MIME type, for example `"audio/mpeg"`.

Default: `none`

item

Describes an item for `rss.feed(items: ..)`.

```
rss.item(
  "Hello, world",
  link: "https://example.com/blog/hello/",
  published: "2026-07-01",
  description: [My _first_ post],
)
```

Parameters

```
item(
  title: str content none,
  link: str none,
  guid: str auto none,
  permalink: bool auto,
  published: datetime str none,
  description: str content none,
  content: str none,
  author: dictionary str none,
  categories: array,
  comments: str none,
  enclosure: dictionary none,
  extra: array
) -> dictionary
```

title `str` or `content` or `none`

The title. It can be `none` if there is a `description`.

link `str` or `none`

The URL of the web page of the item.

Default: `none`

guid `str` or `auto` or `none`

A unique identifier. The default is `link`.

Default: `auto`

permalink `bool` or `auto`

If `true`, `guid` is a URL of the item. The default is `true` for identifiers that start with `http://` or `https://`.

Default: `auto`

published `datetime` or `str` or `none`

The date of publication.

Default: `none`

description `str` or `content` or `none`

A summary or the full text. It can contain HTML source. The function escapes it.

Default: `none`

content `str` or `none`

The full content as HTML source. The function writes it as `<content:encoded>`.

Default: `none`

author `dictionary` or `str` or `none`

The author, as a dictionary `(name: .., email: ..)`, an `atom.author(..)` or a string. If the author has an email, the function writes `<author>`. If not, it writes the name as `<dc:creator>`.

Default: `none`

categories `array`

The categories, as strings or dictionaries `(name: .., domain: ..)`.

Default: `()`

comments `str` or `none`

The URL of the comments on the item.

Default: `none`

enclosure `dictionary` or `none`

A media attachment. Make it with `rss.enclosure(..)`.

Default: `none`

extra `array`

More XML nodes. The function adds them at the end of the item.

Default: `()`

feed

Generates an RSS 2.0 feed. If the feed uses the `atom`, `content` or `dc` namespace, the function declares it automatically.

```
rss.feed(
  title: "My Blog",
  link: "https://example.com/",
  description: "Things I write",
  items: rss.item(
    "Hello, world",
    link: "https://example.com/blog/hello/",
    published: "2026-07-01",
```

```
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0">
  <channel>
    <title>My Blog</title>
    <link>https://example.com/</link>
    <description>Things I write</description>
    <lastBuildDate>Wed, 01 Jul 2026 00:00:00 +0000</lastBuildDate>
    <item>
      <title>Hello, world</title>
      <link>https://example.com/blog/hello/</link>
      <guid isPermaLink="true">https://example.com/blog/hello/</guid>
      <pubDate>Wed, 01 Jul 2026 00:00:00 +0000</pubDate>
    </item>
  </channel>
</rss>
```

Parameters

```
feed(
  title: str content,
  link: str,
  description: str content,
  self-link: str none,
  language: str none,
  copyright: str content none,
  managing-editor: str none,
  web-master: str none,
  published: datetime str none,
  updated: datetime str auto none,
  categories: array,
  generator: str none,
  docs: str none,
  ttl: int none,
  image: str dictionary none,
  items: array,
  tz: str,
  stylesheet: str none,
  namespaces: dictionary,
  extra: array,
  pretty: bool
) -> str
```

title `str` or `content`

The title of the feed. This value is necessary.

Default: none

link `str`

The URL of the website. This value is necessary.

Default: none

description `str` or `content`

A description of the feed. This value is necessary.

Default: none

self-link `str` or `none`

The URL of this feed. The function writes it as `<atom:link rel="self">`. Set it if possible.

Default: none

language `str` or `none`

The language of the feed, for example `"en-us"`.

Default: none

copyright `str` or `content` or `none`

A copyright statement.

Default: none

managing-editor `str` or `none`

The email address of the editor.

Default: none

web-master `str` or `none`

The email address of the person who operates the website.

Default: none

published `datetime` or `str` or `none`

The date of publication of the feed content.

Default: none

updated `datetime` or `str` or `auto` or `none`

The date of the last change (`<lastBuildDate>`). The default is the latest `published` of the items. If `none`, the function does not write it.

Default: `auto`

categories `array`

The categories, as strings or dictionaries `(name: .., domain: ..)`.

Default: `()`

generator `str` or `none`

The software that generated the feed.

Default: `none`

docs `str` or `none`

The URL of the RSS specification.

Default: `none`

ttl `int` or `none`

The maximum time in minutes that a reader can keep the feed in a cache.

Default: `none`

image `str` or `dictionary` or `none`

An image, as a URL or a dictionary `(url: .., title: .., link: .., width: .., height: ..)`. The default `title` and `link` are those of the feed.

Default: `none`

items `array`

The items. Make them with `rss.item(..)`.

Default: `()`

tz `str`

The time zone of all dates. Use `"Z"` for UTC, or an offset, for example `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet. Browsers apply it to the feed.

Default: `none`

namespaces `dictionary`

More namespace declarations `(prefix: uri)` for the elements in `extra`.

Default: `(:)`

extra `array`

More XML nodes. The function adds them at the end of the channel.

Default: `()`

pretty `bool`

If `true`, the function indents the output.

Default: `true`

7.4 sitemap

- url()
- urlset()
- index()

url

Describes a page for `sitemap.urlset(..)`, or a sitemap for `sitemap.index(..)`.

```
sitemap.url(
  "https://example.com/about/",
  lastmod: "2026-05-01",
  alternates: (de: "https://example.com/de/about/"),
)
```

Parameters

```
url(
  loc: str,
  lastmod: datetime str none,
  changefreq: str none,
  priority: int float none,
  alternates: dictionary,
  extra: array
) -> dictionary
```

loc `str`

The canonical URL of the page.

lastmod `datetime` or `str` or `none`

The date of the last change. If the date has no time, the function writes it as `YYYY-MM-DD`.

Default: `none`

changefreq `str` or `none`

The frequency of changes: `"always"`, `"hourly"`, `"daily"`, `"weekly"`, `"monthly"`, `"yearly"` or `"never"`.

Default: `none`

priority `int` or `float` or `none`

The priority of the page, relative to your other pages. Use a number from `0` to `1`.

Default: `none`

alternates `dictionary`

Translations of the page as `(hreflang: url)`, for example `(de: "https://example.com/de/", "x-default": "https://example.com/")`.

Default: `(:)`

extra `array`

More XML nodes. The function adds them at the end of the entry.

Default: `()`

urlset

Generates a sitemap, which lists the pages of a website. A sitemap can contain a maximum of 50 000 URLs. For a larger website, divide the list into more than one sitemap. Then list the sitemaps with `sitemap.index`.

```
sitemap.urlset(
  "https://example.com/",
  sitemap.url("https://example.com/about/", lastmod: "2026-05-01"),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://example.com/</loc>
  </url>
  <url>
    <loc>https://example.com/about/</loc>
    <lastmod>2026-05-01</lastmod>
```

```
  </url>
</urlset>
```

Parameters

```
urlset(
  ..urls: dictionary str array,
  tz: str,
  stylesheet: str none,
  namespaces: dictionary,
  pretty: bool
) -> str
```

..urls `dictionary` or `str` or `array`

The pages, as `sitemap.url(..)` or URL strings. Give them as arguments or arrays.

tz `str`

The time zone of dates that have a time. Use `"Z"` for UTC, or an offset, for example `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet. Browsers apply it to the sitemap.

Default: `none`

namespaces `dictionary`

More namespace declarations `(prefix: uri)` for the elements in `extra`.

Default: `(:)`

pretty `bool`

If `true`, the function indents the output.

Default: `true`

index

Generates a sitemap index, which lists more than one sitemap.

```
sitemap.index(
  "https://example.com/sitemap-pages.xml",
```

```
  sitemap.url("https://example.com/sitemap-posts.xml", lastmod: "2026-07-03"),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<sitemapindex xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <sitemap>
    <loc>https://example.com/sitemap-pages.xml</loc>
  </sitemap>
  <sitemap>
    <loc>https://example.com/sitemap-posts.xml</loc>
    <lastmod>2026-07-03</lastmod>
  </sitemap>
</sitemapindex>
```

Parameters

```
index(
  ..sitemaps: dictionary str array,
  tz: str,
  stylesheet: str none,
  pretty: bool
) -> str
```

..sitemaps `dictionary` or `str` or `array`

The sitemaps, as URL strings or `sitemap.url(loc, lastmod: ..)`.

tz `str`

The time zone of dates that have a time. Use `"Z"` for UTC, or an offset, for example `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet. Browsers apply it to the index.

Default: `none`

pretty `bool`

If `true`, the function indents the output.

Default: `true`

7.5 Dates

- rfc3339()
- rfc2822()

rfc3339

Formats a date as an RFC 3339 timestamp. Atom feeds and sitemaps use this format. If the date has no time, the function uses `00:00:00`.

```
rfc3339("2026-07-01T14:30", tz: "+02:00")
```

↓

```
2026-07-01T14:30:00+02:00
```

Parameters

```
rfc3339(
  dt: datetime str,
  tz: str
) -> str
```

dt `datetime` or `str`

The date.

tz `str`

The time zone of the date. Use `"Z"` for UTC, or an offset, for example `"+01:00"` or `"-0530"`.

Default: `"Z"`

rfc2822

Formats a date as an RFC 2822 timestamp. RSS feeds use this format. If the date has no time, the function uses `00:00:00`.

```
rfc2822("2026-07-01T14:30", tz: "+02:00")
```

↓

```
Wed, 01 Jul 2026 14:30:00 +0200
```

Parameters

```
rfc2822(
  dt: datetime str,
  tz: str
) -> str
```

dt `datetime` or `str`

The date.

tz `str`

The time zone of the date. Use `"Z"` for UTC, or an offset, for example `"+01:00"` or `"-0530"`.

Default: `"Z"`